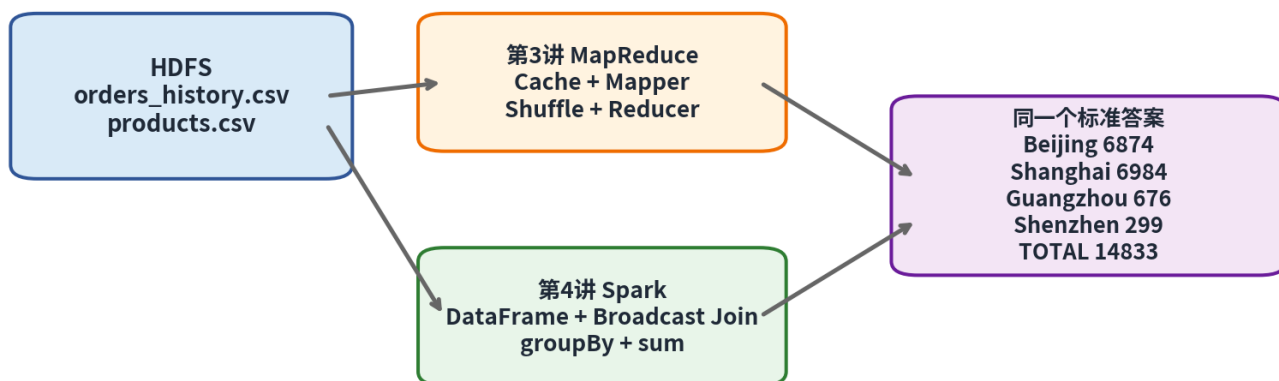


大数据综合项目实战

第 4 讲：用 Spark DataFrame 统计各城市 PAID 订单销售额

同一业务、同一数据、同一标准答案：从 MapReduce 迁移到 Spark

同一个业务问题：MapReduce 与 Spark DataFrame 两种实现



固定业务与数据，只改变计算引擎，才能真正看清两种分布式计算模型的差别。

本讲最终目标

从 HDFS 读取 orders_history.csv 和 products.csv；使用显式 Schema、Broadcast Join、DataFrame Transformation 和 groupBy 聚合；得到 TOTAL=14833；把结果写入 HDFS Parquet 并重新读回验证。

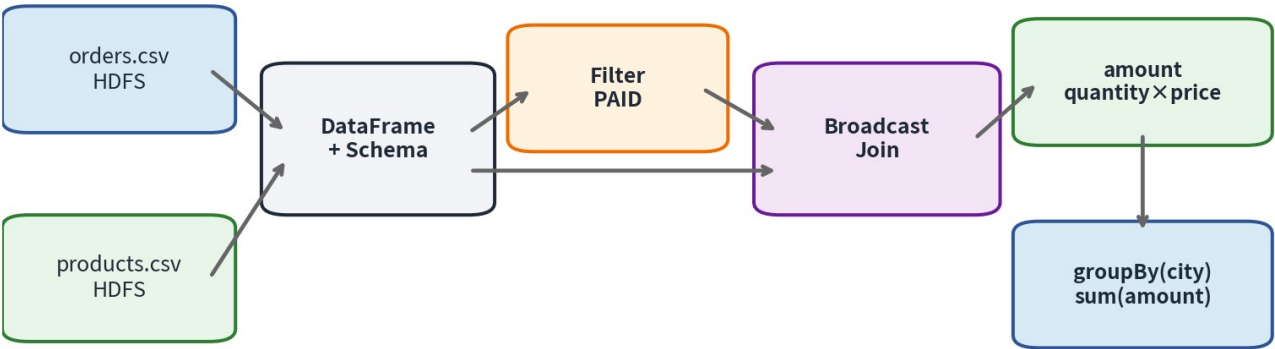
统一环境：Ubuntu 22.04 + Hadoop 3.3.5 + YARN；Windows 端继续使用 VS Code Remote SSH。Spark 使用现有可用的 Spark 3.x / PySpark 环境，本讲先用 spark-submit --version 检查，不假设固定安装路径。

1. 为什么第 4 讲故意重复第 3 讲的业务问题？

如果我们同时换“数据、业务问题、计算引擎”，你很难判断到底是哪一部分造成了差异。因此第 4 讲保持订单数据、商品数据、过滤规则和标准答案全部不变，只把 MapReduce 换成 Spark DataFrame。

对比项	第 3 讲 MapReduce	第 4 讲 Spark DataFrame
小表关联	Distributed Cache + setup() HashMap	broadcast(products) + join
过滤	Mapper if(status==PAID)	filter(col("status")== "PAID")
金额计算	Mapper quantity × price	withColumn("amount", ...)
按城市聚合	Shuffle + Reducer	groupBy("city").sum(...)
结果存储	文本 part-r-00000	Parquet 目录
执行结构	MapTask / ReduceTask	Job / Stage / Task / Executor

第4讲 Spark DataFrame 数据流



关键：Spark 没有消灭 Join 和 Shuffle，而是用更高层的 DataFrame / SQL 表达它们。

这一讲不是“语法课”

真正目标是把 DataFrame 代码重新对应到底层：Partition、Join、Shuffle、Stage、Task 和 Executor。

2. 标准答案仍然固定：TOTAL=14833

第 1 讲已经手工验证，第 3 讲 MapReduce 也已经算过。Spark 版本必须得到完全相同的业务结果。

城市	期望销售额
Beijing	6874
Guangzhou	676
Shanghai	6984
Shenzhen	299
TOTAL	14833

验收原则

Spark Application 状态为 FINISHED/SUCCEEDED 只能证明“系统执行完成”，不能证明业务结果正确。TOTAL 和四个城市都必须对上。

3. 上机前：从虚拟机、VS Code、服务、Spark 环境开始检查

每次课都默认：虚拟机刚打开，HDFS/YARN 服务未启动。不要跳过服务检查。

```
# 1) 确认你在 Ubuntu 虚拟机，而不是 Windows PowerShell
whoami
hostname
pwd

# 2) 检查 Java / Hadoop / Spark 命令
java -version
hadoop version | head
spark-submit --version

# 3) 先看当前 Hadoop 进程
jps
```

接着启动 HDFS 与 YARN：

```
start-dfs.sh
start-yarn.sh

# 再检查
jps

# HDFS 是否处于安全模式外
hdfs dfsadmin -safemode get

# YARN 是否看到 NodeManager
yarn node -list
```

预期 jps

至少应看到 NameNode、DataNode、SecondaryNameNode、ResourceManager、NodeManager、Jps。单节点教学环境通常所有角色都在同一台 VM。

4. 先确认第 2 讲 HDFS 输入还在

```
hdfs dfs -ls /bigdata-capstone/raw/orders
hdfs dfs -ls /bigdata-capstone/raw/products

hdfs dfs -cat /bigdata-capstone/raw/orders/orders_history.csv
hdfs dfs -cat /bigdata-capstone/raw/products/products.csv
```

确认两张表都能看到表头与数据。特别检查 orders_history.csv 共有 12 条业务记录，products.csv 共有 5 条商品记录。

为什么 Spark 也从 HDFS 读？

综合项目要训练“存储层与计算层分离”：HDFS 负责保存项目数据，MapReduce 和 Spark 都只是读取同一份 HDFS 数据来计算。

5. 在 VS Code 中创建第 4 讲目录

```
cd ~/bigdata-capstone
mkdir -p spark/lecture4
cd spark/lecture4
pwd
ls -lah
```

把代码包中的 04_city_paid_sales_spark.py 放到该目录。第一次建议在 VS Code 中打开源码，完整阅读一遍，再运行。

6. 为什么本讲不用 inferSchema，而是手工定义 Schema？

Spark 可以 inferSchema，但科研/生产式习惯更强调“数据契约”：哪些字段是字符串、哪些必须是整数，应由我们显式规定。这样异常数据不会悄悄把类型推断成另一个结果。

```
orders_schema = StructType([
    StructField("order_id", StringType(), False),
    StructField("order_time", StringType(), False),
    StructField("user_id", StringType(), False),
    StructField("product_id", StringType(), False),
    StructField("quantity", IntegerType(), False),
    StructField("status", StringType(), False),
    StructField("city", StringType(), False),
])
```

读取 HDFS CSV:

```
orders = (
    spark.read
    .option("header", True)
    .schema(orders_schema)
    .csv("/bigdata-capstone/raw/orders/orders_history.csv")
)

orders.printSchema()
print("orders rows =", orders.count())
```

第一层证据

orders.printSchema() 必须显示 quantity 为 integer；orders.count() 应为 12。

7. DataFrame 第一步：先 Filter，再 Join

先过滤 PAID，可以让后面的 Join 少处理 2 条无关记录。数据量很小时差别看不出来，但在大数据里“尽量在 Shuffle / Join 前减数据”是非常重要的习惯。

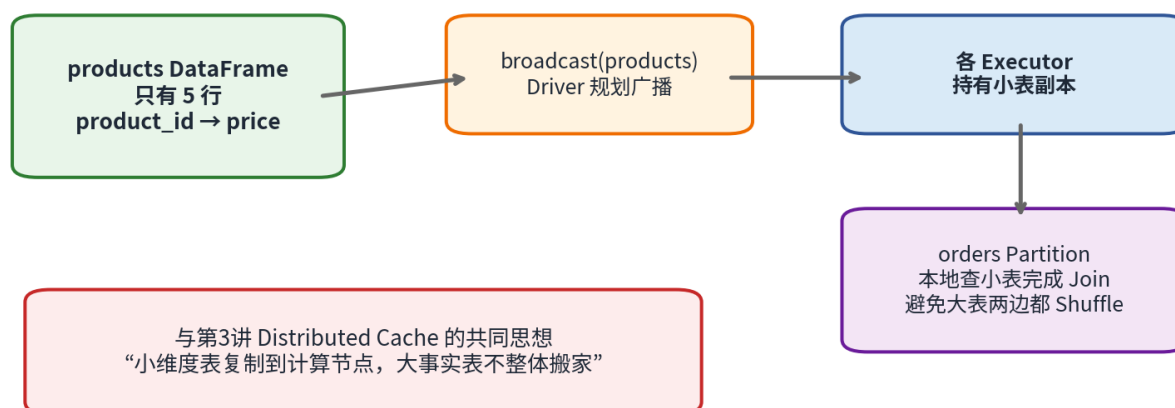
```
paid = orders.filter(col("status") == "PAID")
print("PAID rows =", paid.count()) # expected 10
```

products.csv 只有 5 行，因此本讲明确使用 Broadcast Join:

```
joined = (
    paid.join(
        broadcast(products.select("product_id", "unit_price")),
        on="product_id",
        how="inner",
    )
    .withColumn("amount", col("quantity") * col("unit_price"))
)

print("joined rows =", joined.count()) # expected 10
```

Broadcast Join : Spark 版的“小表复制到各执行节点”



区别：Spark 的广播 Join 由执行计划与 DataFrame API 组织；MapReduce 中我们手工在 Mapper setup() 里加载缓存文件。

为什么 Broadcast Join 重要？

如果两边都按 product_id 做普通 Shuffle Join，大表也要跨网络重分区。广播小表可以让订单大表尽量留在原 Partition，本地完成查表。

8. 按城市聚合：真正的 Shuffle 在这里出现

```
result = (
    joined.groupBy("city")
        .agg(spark_sum("amount").alias("sales_amount"))
        .orderBy("city")
)

result.show(truncate=False)
```

groupBy("city") 的业务含义与第 3 讲完全一样：同一个 city 的 amount 必须“见面”。因此 Spark 必须把相同 city 的记录重新分区到一起，这就是 Shuffle。

不要误解 “Spark 在内存里所以没有 Shuffle”

Spark 只是比传统 MapReduce 更灵活地组织执行。只要业务要求跨 Partition 按 Key 聚合，网络数据重分布仍然存在。

9. 业务验收：四城金额和 TOTAL 必须同时验证

```
total = result.agg(
    spark_sum("sales_amount").alias("total")
).collect()[0]["total"]

print("TOTAL =", total)
```

本讲教学数据只有 4 行聚合结果，因此 collect() 到 Driver 是安全的。不要把这个习惯机械推广到海量原始 DataFrame；对百万/亿级结果直接 collect() 可能撑爆 Driver。

检查项	期望
orders rows	12
products rows	5
PAID rows	10
joined rows	10
Beijing	6874
Guangzhou	676
Shanghai	6984
Shenzhen	299
TOTAL	14833

代码中为什么直接 assert?

综合项目开始培养“自动验收”习惯。结果错时程序应该明确失败，而不是只打印一堆数字让人肉眼猜。

10. 第一次把 DataFrame 代码重新翻译回底层：explain()

执行 `result.explain(mode="formatted")`。不同 Spark 小版本输出细节可能略有区别，所以不要死背完整文本，只抓本讲四类关键词。

本讲 explain() 只抓 4 个关键词



看到 **Exchange** 时要立即想到：这里发生了跨 Partition 的数据重分布，通常是性能成本边界。

```
result.explain(mode="formatted")
```

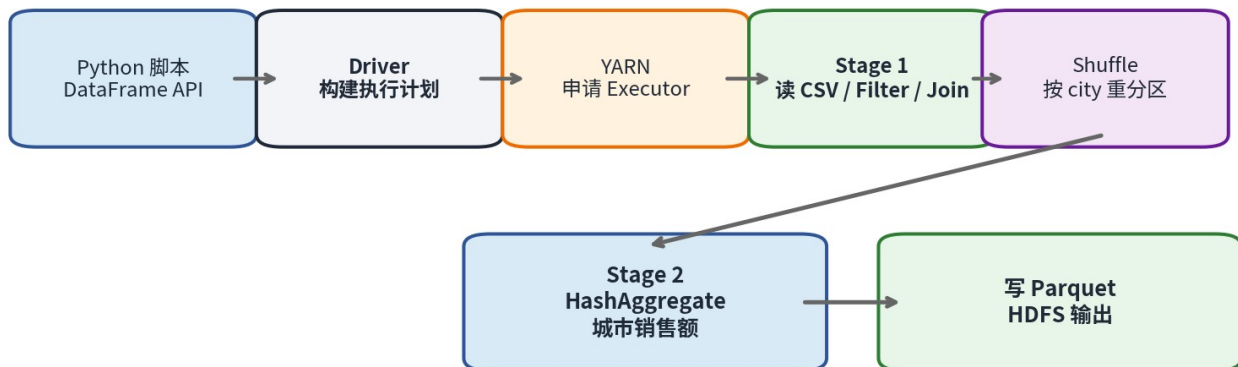
典型情况下你会看到：读取 CSV 的 Scan、BroadcastHashJoin、聚合前后的 Exchange / HashAggregate，以及可能的 AdaptiveSparkPlan。

最重要的阅读习惯

看到 Exchange：问“为什么这里必须重分区？”；看到 BroadcastHashJoin：问“哪一边被广播，为什么可以广播？”；看到 Aggregate：问“状态/中间聚合在哪里发生？”

11. PySpark 脚本在 YARN 上到底怎样运行？

从 PySpark 代码到 YARN 上真正运行



Spark 的“高级 API”最终仍会落到 Partition、Task、Shuffle、Executor 与网络/磁盘。

你在终端运行 spark-submit 后，Driver 解析 DataFrame 依赖并向 YARN 申请 Executor。Executor 在各 Partition 上执行 Task。groupBy 引入跨 Partition 重分布，通常形成新的 Stage 边界。

Spark 概念	在本讲中的具体含义
Driver	运行 Python 主程序，构建并协调 Job
Executor	真正执行 Partition 上的 Task
Partition	订单 DataFrame 的分布式数据分片
Task	处理一个 Partition 的执行单元
Stage	一组不跨 Shuffle 边界的 Task
Shuffle / Exchange	把相同 city 的记录重新组织到一起

12. 保姆式运行：spark-submit 提交到 YARN

先在 VS Code 远程终端进入第 4 讲目录。

```
cd ~/bigdata-capstone/spark/lecture4
ls -lh 04_city_paid_sales_spark.py
```

```
# 先看 Spark 版本，确认命令可用
spark-submit --version
```

然后提交到 YARN。教学虚拟机使用非常保守的资源参数：

```
spark-submit --master yarn --deploy-mode client --num-executors 1 --executor-cores 1 --executor-memory 1g
04_city_paid_sales_spark.py
```

为什么用 client mode?

零基础学习阶段让 Driver 日志直接出现在当前 VS Code 终端，排错更直观。生产部署不等于必须使用 client mode。

运行时不要只盯最后一行。你应该观察：Spark Application ID、YARN 资源申请、Stage/Task 进度、四城市结果、TOTAL、[PASS] 验收，以及 Parquet 写入提示。

13. 为什么第 4 讲输出 Parquet，而不是再写 TXT？

第 3 讲 MapReduce 输出文本很适合人工查看；Spark 分析链更常把中间/结果数据保存成 Parquet。Parquet 是列式格式，保留 Schema，更适合后续 Spark SQL / DataFrame 再读取。

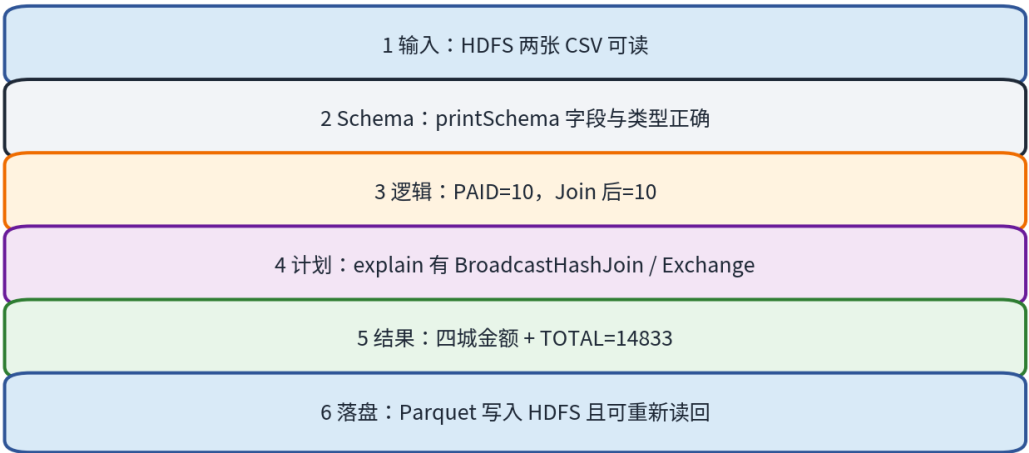
```
# 看 HDFS 输出目录
hdfs dfs -ls /bigdata-capstone/output/spark/city_paid_sales_parquet

# Parquet 是二进制列式文件，不要用 hdfs dfs -cat 硬看内容。
# 本讲 Python 程序会自己重新 spark.read.parquet(...) 回读并 show()。
```

一个重要区别
CSV/TXT 更适合人肉阅读；Parquet 更适合计算引擎高效读取。不能因为 hdfs dfs -cat 看不懂 Parquet 就认为文件损坏。

14. 本讲六层证据链：从“能跑”升级到“可验证”

第4讲验收：六层证据链



如果这六层都通过，本讲才算真正完成。尤其是最后一层“写入以后重新读回”非常重要，它验证的不只是内存中的 result，而是 HDFS 中最终产物。

15. 同一业务的 MapReduce vs Spark：现在应该能看出本质差别

问题	MapReduce 第 3 讲	Spark 第 4 讲
关联小表	手工 Cache + HashMap	broadcast + join
数据过滤	map() 中 if	filter()
派生金额	map() 中计算	withColumn()
按城市重组	框架 Shuffle 到 Reducer	groupBy 触发 Exchange/Shuffle
聚合	reduce() 手工 for 求和	agg(sum())
执行计划可见性	主要看 Job/Counters/日志	explain + Spark UI
输出	Text	Parquet + Schema

Spark 为什么看起来代码少很多？
不是因为分布式问题消失了，而是 DataFrame API 把“数据依赖和关系运算”描述得更高层，Catalyst/Spark Runtime 帮你

组织很多底层执行细节。

16. 常见错误：这次优先从 DataFrame / Spark 层排查

现象	第一检查项	常见原因
spark-submit: command not found	Spark 环境	PATH / 安装未配置
YARN 提交失败	ResourceManager / NodeManager	YARN 未启动或资源不足
AnalysisException 找不到列	Schema / 列名	拼错 product_id / unit_price
PAID rows 不是 10	输入数据 / status	CSV 变更或过滤条件错误
joined rows < 10	产品维表	商品缺失导致 inner join 丢行
TOTAL 不等于 14833	金额计算 / Join	quantity/price 类型或业务逻辑错误
Parquet 输出存在但无法 cat	文件格式理解	Parquet 是二进制列式格式，不应 cat

固定排错顺序

服务 → HDFS 输入 → Spark 命令 → Schema → 每步 count → Join → 聚合 → explain → HDFS 输出 → 业务标准答案。

17. 故意制造 3 个错误：训练 Spark 排错能力

实验 A：把 status 列误写成 state

预期：Spark 抛出未解析列/AnalysisException。学会把“列名错误”与 YARN/网络错误区分开。

实验 B：删除 products 中 P002 后写到测试路径

预期：使用 inner join 后 joined rows 会小于 PAID rows，最终 TOTAL 变小。学会把 Join 行数作为关键业务证据。

实验 C：临时去掉 broadcast(...)

预期：业务结果仍应正确，但 Physical Plan 的 Join 策略可能变化。比较 explain()，理解“正确性”和“性能执行计划”是两件事。

18. 迁移练习

题号	问题
1	如果统计“每个商品 PAID 销售额”，groupBy 应改成什么？
2	如果统计每个城市 PAID 订单数，还需不需要 products.csv？为什么？
3	如果 products.csv 变成 200 GB，还应该 broadcast 吗？
4	为什么 groupBy(city) 通常会产生 Exchange？
5	为什么本讲 result.collect() 可以接受，但不能对海量订单明细直接 collect()？

19. 参考答案

题号	答案
1	groupBy("product_id"), 再 sum(amount)。
2	不需要。订单数只依赖 status 和 city, 不需要单价。
3	不应该。广播大表会给 Driver/Executor 和网络带来巨大压力, 应采用其他 Join 策略。
4	因为同城市数据可能分散在不同 Partition, 聚合前必须按 city 重新分区。
5	本讲最终只有 4 行城市结果; 海量明细 collect() 会把所有数据拉到 Driver, 可能 OOM。

20. 本讲最终验收清单

- 能启动 HDFS/YARN, 并确认 Spark 命令可用。
- 能从 HDFS 读 orders 与 products 两张 CSV。
- 能解释为什么显式 Schema 比盲目 inferSchema 更可控。
- orders=12、products=5、PAID=10、joined=10。
- 能解释 broadcast(products) 为什么适合 5 行小表。
- 能解释 groupBy(city) 为什么仍然需要 Shuffle。
- 四城市结果与 TOTAL=14833 完全一致。
- 能在 explain 中识别 Scan / BroadcastHashJoin / Exchange / HashAggregate。
- 能使用 spark-submit --master yarn 提交作业。
- Parquet 已写入 HDFS, 并被 Spark 重新读回验证。
- 能说出同一业务下 MapReduce 与 Spark 的核心开发模型差异。

本讲最低闭环能力

不依赖辅助脚本, 你能完成: 服务检查 → HDFS 输入确认 → DataFrame 读取 → Schema → Filter → Broadcast Join → amount → groupBy → explain → YARN 提交 → Parquet → 标准答案验收。

21. 课后停止服务

```
# 确认没有仍在运行的 YARN Application
yarn application -list

stop-yarn.sh
stop-dfs.sh
jps
```

22. 下一讲预告：离线分析不只算一个指标

第 5 讲会在同一批 HDFS 数据上扩展 Spark 离线分析: 用户表、商品表、订单表三表 Join, 生成城市销售额、品类销售额、Top 商品、VIP/Normal 用户消费等多个指标, 并把结果统一写成 Parquet。到那时我们会开始建立真正的“离线分析层”, 而不是只完成一道聚合题。

为什么第 5 讲才扩展业务?

第 4 讲先固定单一指标完成 MapReduce→Spark 的可控迁移; 确认你理解 Spark 的执行闭环后, 再增加 Join 与指标复杂度。